Examiner's Report

Master's thesis (M.Sc.A.), Polytechnique Montréal

Student:	William Zhang (2362162), Electrical Engineering
Title:	*A Lightweight Canonicalization Engine in C for Disciplined Convex Programming*
Supervisor:	Antoine Lesage-Landry
Examiner:	Antoine Legrain, Professor, Department of Mathematics and Industrial Engineering
Date:	September 24, 2026

Partie 1 : Valeur scientifique

Summary of the work

The thesis addresses the compilation (canonicalization) cost of convex problems in CVXPY, that is, the conversion of expression trees into the sparse matrices (P, c, A, b) accepted by a conic solver. The student takes the view that this step is a sparse derivative computation (A is the Jacobian of the constraints, P the Hessian of the objective) and proposes `SparseDiffEngine`, a C library that (i) splits the computation into a symbolic pass, run once per problem family, and a numeric pass, repeated at every solve, and (ii) introduces two matrix representations (*permuted dense* and *stacked permuted dense*) that keep the dense blocks of the data intact and process them with BLAS kernels. A new CVXPY backend, `diffengine`, is built on this library and compared with the three existing backends on CVXPY's public benchmark suite of 25 problems. The first canonicalization is the fastest on 21 of the 25 problems (2.8× geometric-mean speedup) and parametric re-solves run at times comparable to CVXPY's cached DPP tensor.

Assessment

1. **Originality and positioning.** The topic is original and relevant: model compilation is, as the conclusion puts it, a blind spot of the community, even though it sometimes dominates total time for parametric or large-scale problems. Recasting matrix stuffing as structured sparse differentiation is elegant and well argued. The student honestly acknowledges (§2.4, §3) that the computation is mathematically equivalent to the composition of linear operators already performed by CVXPY and that the contribution lies in the implementation; this lucidity is appreciated. The research gap is clearly identified and summarized in Table 2.2, which compares AMPL, JuMP, CasADi and CVXPY very convincingly.

2. **Coherence and structure.** The structure is exemplary: introduction (problem, canonicalization, derivatives, matrix computations), a literature review organized around the question "how do modelling languages obtain their coefficients?", presentation of the engine, results, conclusion. Each chapter announces its plan and follows it, and the running example (1.5) is carried through Chapters 1, 2 and 3, which greatly eases reading. Chapters 1 and 2 form a remarkably clear exposition of the knowledge acquired (DCP,

CVXPY reductions, sparse AD and graph colouring, storage formats, `.nl` files, MOI), at and beyond the level expected of a master's thesis.

3. **Use of the literature.** The bibliography (89 references) is rich, well chosen, and covers both the historical sources (Markowitz 1957, Gustavson 1978, Curtis–Powell–Reid 1974, Fourer 1983) and very recent work (Hill and Dalle 2025, CVXPY 1.9, Clarabel). The student clearly commands the literature of several sub-fields (modelling languages, algorithmic differentiation, sparse linear algebra) and knows how to connect them. Several directly related works are missing, however, and their absence weakens the positioning of the thesis:

   (a) *Diamond and Boyd, "Matrix-free convex optimization modeling" (2016; arXiv:1506.00760).* This is the most important omission. The creators of CVXPY pose exactly the question of the thesis, namely how to deliver the problem data to the solver without materializing generic sparse operators, and answer it with the opposite solution: do not form a matrix at all, but represent the problem as a graph of abstract affine operators (FAOs) evaluated through forward and adjoint products, preserving Kronecker structure and dense blocks. The thesis instead materializes the matrix, but in formats that retain the structure. The two approaches must be confronted: why materialize? The answer is solid (production conic solvers, including Clarabel used in Chapter 4, require an explicit matrix for factorization; the matrix-free approach never went beyond a research fork because it assumed modified solvers and only pays off for very large problems built on fast transforms), but it must be written down. The permuted dense matrices can in fact be presented as an intermediate form between the abstract FAO and generic CSR: a materialized matrix that keeps the operator structure (selectors plus a dense block). This paragraph would, in my view, be the most valuable addition to the final version.

   (b) *CVXcanon (Miller, Quigley and Zhu, 2015).* The direct ancestor of `cvxcore`, which the thesis describes in detail without citing the work that created it.

   (c) *CVXGEN (Mattingley and Boyd, 2012).* The first answer to the parametric problem through code generation, and the ancestor of CVXPYgen [66], which is cited.

   (d) *Convex.jl, version 0.16 (2024).* The competing DCP language in Julia rewrote its internals around a `SparseTape` representation that accumulates affine expressions directly as sparse matrices during construction; this is another answer to the same problem within the same paradigm. Convex.jl is cited only through its 2014 paper [20].

   (e) *Block-sparse Jacobians outside convex optimization.* Keeping dense blocks inside a sparse matrix and processing them with `dgemm` is an old and well-established idea: BSR formats, element-wise assembly in finite elements, and above all the block-sparse Jacobians of Ceres Solver or g2o in bundle adjustment, which are very close to the stacked permuted dense matrix of §3.2.3. Likewise, the symbolic/numeric

separation belongs first to the tradition of sparse factorizations (symbolic analysis then numeric factorization, Davis [57]) and to Gustavson [56], both cited by the thesis, before being that of the NLP solver interfaces to which the thesis attributes it. The point is not to take originality away from the work, but to state honestly what is borrowed and what is new.

(f) *Pyomo.* Its persistent solver interfaces (APPSI) update coefficients in place without rewriting an `.nl` file, which would qualify the AMPL/Pyomo row of Table 2.2.

4. **Choice of methods and experimental rigour.** The evaluation methodology is sound: public benchmark suite, exclusion of solver time, Dolan–Moré performance profiles, published scripts and raw timings, CVXPY version and hardware specified, code available as a pull request. The results are reported honestly (the four problems on which `diffengine` is slower are named and discussed, the higher peak memory is acknowledged). This is nevertheless where the main weakness of the thesis lies: the analysis of the results is thin relative to the quality of the rest of the document (Chapter 4 is six pages long). All of the following points are required corrections for the final version, and I will also raise them at the defence:

(a) *Attribution of the speedup (ablation).* The thesis identifies two limitations of CVXPY's backends (§2.1.3: generic sparse libraries, dense blocks stored as sparse) and two contributions (symbolic/numeric split, permuted dense representations). The experiments, however, do not tell us what share of the 2.8× factor comes from the dense representations, what share from the two-pass split, and what share simply from a leaner C implementation (global rather than local layout, no materialization of views, no parameter tensor). A variant of `diffengine` with the dense representations disabled (everything in CSR), run on the 25 problems, would settle the question. Without it, the claimed contribution is not isolated, which is insufficiently rigorous for a research thesis.

(b) *Demonstration of the central claim on non-DPP problems.* The strongest argument of the thesis is that caching does not require the DPP grammar. Yet the seven parametric problems of Table 4.2 are all DPP. No experiment shows a re-solve of a parametric problem that does *not* conform to DPP (for instance a product of two parameters), the very case in which CVXPY has no alternative to full recomputation. A single example would suffice to support the first observation of §4.3.

(c) *Repetitions and variability.* Each measurement is run once ("runs `get_problem_data` once"). For times of the order of a second this is acceptable; for the re-solves of Table 4.2 (0.001 s versus 0.003 s) and the `Param*MatrixStuffing` problems (0.14 s versus 0.18 s), differences of a few milliseconds cannot be interpreted without repetitions and standard deviations, especially on a laptop.

(d) *Instance sizes and scaling.* The dimensions of the 25 problems (n, m, number of nonzeros, number of parameters) are not reported, which prevents the reader from

judging the difficulty of the instances and how the speedup evolves with size. A column or an appendix would suffice; a scaling experiment on one problem (time as a function of n) would be a plus.

(e) *Profiling.* No breakdown of time is given (Python→C conversion, symbolic pass, numeric pass, BLAS calls). Such a breakdown would in particular explain the two underperformances (`SDPSegfault1132`, `ConvexPlasticity`), which are dealt with in one sentence, and would indicate where to direct future effort.

(f) *Correctness validation.* Chapter 4 does not state that the data (P, c, A, b) produced by `diffengine` were compared numerically with those of the other backends on the 25 problems. The test suite mentioned in §3.1 probably does this; it should be written.

5. **Analysis and interpretation.** The three observations of §4.3 are pertinent and well formulated. Two remarks. First, the third observation ("`diffengine` canonicalizes faster on 24 of the 25 problems") appears to contradict §4.1 and the abstract (21 of 25); it is in fact a comparison with `cvxcore` alone, which should be made explicit. Second, the analysis of the two problems on which the tensor re-solves faster (SVM, FactorCovariance) is correct and points to a structural limitation: the numeric pass recomputes the whole constant part even when a single scalar changes. The obvious remedy, propagating a "modified subtree" flag so that only the nodes depending on the updated parameters are re-evaluated, is neither discussed nor listed as future work; it deserves to be, since it would probably make `diffengine` at least as fast as the tensor in every case.

6. **Theoretical analysis.** The thesis contains no complexity analysis, even elementary, of the proposed representations: for instance the memory cost of an $m' \times n'$ block in permuted dense form ($m'n' + m' + n'$) versus CSR ($2m'n' + m + 1$), or the cost of the products (3.2) as a function of block sizes rather than global dimensions, asserted in §3.3.1 but not quantified. A few lines would suffice and would give a more analytical character to a Chapter 3 that is otherwise purely descriptive.

7. **Contribution to knowledge and to technological development.** The technological contribution is substantial and real: an open, tested C library with Python bindings, integrated into CVXPY through a pull request that has since been merged, and that could replace the default backend of a package used by tens of thousands of users. The student is moreover a co-author of two recent CVXPY preprints [27, 34], evidence of successful integration into the research community. The contribution to knowledge is more modest and architectural in nature: neither the symbolic/numeric split nor dense blocks inside a sparse matrix are new in themselves (see point 3), but their combination applied to the matrix stuffing of a DCP language, caching without the DPP grammar, and the comparison with the parameter tensor on a public suite have not, to my knowledge, been done before. The view that "matrix stuffing is structured sparse differentiation" is also an illuminating way of connecting literatures that ignore each other. The thesis

itself acknowledges the mainly software nature of its contribution; it would gain from being equally explicit about the age of its two basic ideas.

Partie 2 : Qualité de la présentation du document

1. **Title.** The title is precise and readable. One nuance: the engine performs only the back end of canonicalization (matrix stuffing, or derivative assembly), the front-end reductions remaining in CVXPY. The text says so clearly (§1.1, §3.4); the title may be perceived as slightly broader than the content, without this being a problem.

2. **Quality of the writing.** The English is excellent: short sentences, precise vocabulary, very few errors. The pedagogical progression is remarkable, notably the transition between §2.1 (what CVXPY does) and §3 (what the engine does) and the systematic return to example (1.5). Chapters 1 and 2 could serve as teaching material. The student transparently declares the use of language models as writing assistants; the result is clearly mastered and coherent.

3. **Illustrations and tables.** The figures are careful and useful: Figures 2.3, 3.2, 3.6, 3.7 and 3.8 explain mechanisms that text alone would leave obscure, and Figure 3.5 (double dispatch) elegantly summarizes §3.3.1. Tables 2.1, 2.2, 3.1 and 3.2 are well designed. The performance profiles (Figure 4.1) are legible. One minor reservation: Figure 3.5 relies on colour to distinguish the four kernel families, which does not survive black-and-white printing; a pattern or a letter per family would fix it.

4. **References.** The style is uniform (IEEE) and the references are complete on the whole. A few minor corrections are listed in the appendix.

5. **French abstract.** The résumé is correct and faithful. “Librairie” (p. vii) is an anglicism; “logiciel” should be used.

6. **Minor internal consistency.** The ranking of the contributions varies slightly between sections: §1.5 lists them without hierarchy, the introduction of Chapter 3 puts the two-pass split first, and §3.5 declares that the “main contribution” is the two matrix classes. This should be harmonized.

Partie 3 : Autres commentaires

Overall judgement

With respect to the aim of a master’s thesis at Polytechnique (to present the knowledge acquired, to demonstrate an aptitude for research, and to communicate results), two of the three are met at a level well above average: the exposition of knowledge (Chapters 1 and 2) is of remarkable clarity and depth, and the communication is excellent. The software

realization is substantial, and its merger into CVXPY by the maintainers is a real validation.

The demonstration of research aptitude is, however, incomplete, and this is why my judgement is more reserved than the quality of the writing would suggest. The experimental chapter does not meet the level of rigour expected of a research thesis. The claimed contribution is never isolated: no ablation tells us whether the speedup comes from the dense representations, from the two-pass split, or simply from a leaner C implementation. The central claim, caching without the DPP grammar, is not tested on a single non-DPP problem, that is, on the very case that motivates it. Every measurement is a single run, with no variance, on a laptop. Instance sizes are not reported, no time breakdown explains the problems on which the engine underperforms, and correctness against the existing backends is not stated. The thesis shows *that* the engine is faster; it does not establish *why*, and a research thesis must do the second. A second, lesser reservation concerns positioning: the matrix-free approach of Diamond and Boyd, which answers the same question with the opposite solution, is neither cited nor discussed, and the antecedents of the two basic ideas are not named, so that the thesis appears more novel than it is.

I want to be explicit that, on the strength of the experimental chapter alone, this thesis is borderline for major revisions, and I hesitated before not requesting them. What tipped the balance is that the weaknesses are correctable at modest cost, and that the infrastructure to correct them (public suite, scripts, an engine in which the dense representations can be switched off) already exists. All the items of Part 1, points 3 and 4, are therefore required for the final version, not suggested. A spot comparison with Convex.jl on two or three problems would be a welcome extension but is not required; comparisons with CasADi, AMPL, JuMP or CVXGEN are not required either, for the reasons the thesis itself gives in §3.5.

Questions for the defence

1. The matrix-free approach of Diamond and Boyd avoids forming the matrix A altogether, replacing it with a graph of abstract affine operators. Why materialize dense blocks rather than leave them as operators? In which cases is your approach preferable, and in which is the reverse true?

2. If the permuted dense representations are disabled and everything is computed in CSR inside `SparseDiffEngine`, what fraction of the 2.8× speedup remains?

3. How does re-solving behave on a non-DPP parametric problem, for instance with a product of two parameters or a parameter in a non-affine position?

4. Why not re-evaluate, on a parameter update, only the subtrees that depend on it? What would be the implementation difficulty?

5. The global layout (one Jacobian with `n_vars` columns per node) is chosen for simplicity. In which situations (very large numbers of variables, deep trees) does it become a handicap relative to CVXPY's local layout, in particular for memory?

6. The `SparseDiffEngine` repository presents it as the differentiation engine of the DNLP extension (Cederberg, Zhang, Nobel and Boyd [34]). What part of the library pre-existed the thesis or was developed for DNLP, and what part (affine atoms, permuted dense classes, conic converter, `diffengine` backend) is specific to this work?

7. What is the strategy for build dependencies (BLAS) and binary distribution of `sparsediffpy`, given that the removal of `cvxcore` is motivated in part by the wish to stop shipping C++ binaries?

Rapport synthèse

The presentation of the thesis is recommended, but narrowly: the experimental chapter, taken alone, would justify major revisions, and the recommendation is conditional on all the following corrections being fully incorporated into the final version, which I will check before approving the deposit. Required: (i) a discussion of the matrix-free approach of Diamond and Boyd and of the other references of Part 1, point 3, and an explicit delineation of the thesis's own contribution relative to DNLP; (ii) the ablation study of Part 1, point 4(a), on the full benchmark suite; (iii) at least one non-DPP parametric problem in the re-solve experiments; (iv) repeated measurements with means and standard deviations; (v) the dimensions of every instance; (vi) a time breakdown by phase explaining the underperforming problems; (vii) an explicit statement of the correctness check against the existing backends. The minor corrections of the appendix should also be made.

Appendix: minor corrections

- Chapter 2: add and discuss the missing references of Part 1, point 3: S. Diamond and S. Boyd, "Matrix-free convex optimization modeling", in *Optimization and Its Applications in Control and Data Sciences*, Springer, 2016 (arXiv:1506.00760); J. Miller, P. Quigley and J. Zhu, "CVXcanon: Automatic canonicalization of disciplined convex programs", Stanford, 2015; J. Mattingley and S. Boyd, "CVXGEN: A code generator for embedded convex optimization", *Optimization and Engineering*, 13(1), 2012; the release notes of Convex.jl 0.16; and an antecedent for block-sparse Jacobians (e.g. Ceres Solver, S. Agarwal and K. Mierle).

- p. 3 and p. 46: attribute the symbolic/numeric separation first to sparse factorizations [57] and Gustavson [56], then to NLP interfaces.

- Table 2.2: qualify the AMPL/Pyomo row in light of Pyomo's persistent interfaces.

- §1.5 and §3.1: explicitly delineate the thesis's own contribution relative to the engine developed jointly for DNLP [34], of which `SparseDiffEngine` is also the differentiation engine.

- p. 48: update the status of PR #3448, merged into CVXPY on 9 September 2026

(`DIFFENGINE` backend, opt-in, parameter-free problems only); state that the parametric path is not yet integrated.

- Chapter 4: mention that the student himself contributed to the SCIPY and COO backends against which `diffengine` is compared.
- p. vii, Résumé: "librairie" → "logiciel".
- p. 52, §4.3, third observation: write "faster than `cvxcore` on 24 of the 25 problems" to avoid the apparent contradiction with "fastest on 21 of the 25" (p. 49 and abstract).
- p. 46, §3.5: harmonize the designation of the "main contribution" with §1.5 and the introduction of Chapter 3.
- p. 48, Chapter 4: state explicitly that the data produced by `diffengine` were verified numerically against those of the other backends.
- p. 48: give the number of repetitions per measurement (or add repetitions) and the BLAS implementation used (Accelerate? OpenBLAS?), which directly affects the dense kernels.
- Table 4.1 or appendix: add the dimensions of the 25 problems (n, m, nnz, number of parameters).
- Figure 3.5: do not rely on colour alone to distinguish the kernel families.
- References [41], [42], [43], [76], [88]: "p." → "pp." for page ranges.
- Reference [60] (Eigen): add the URL (`https://eigen.tuxfamily.org`) and the version used.
- Reference [72] (CppAD): incomplete; add the URL or report.
- Reference [21] (MOSEK): add the documentation URL.
- Reference [17] (CVX): the cited version (2.0, 2012) is old; cite the current version or indicate that this is the canonical reference.